ScholarPulse: AI-Enabled Academic Knowledge Hub

Milestone 1 (MS-1): Problem Statement and Design Document (Revised)

Course Project 2026 | Theme 4: Knowledge Hub | Team of 4

1 Overview

- **Website name:** ScholarPulse

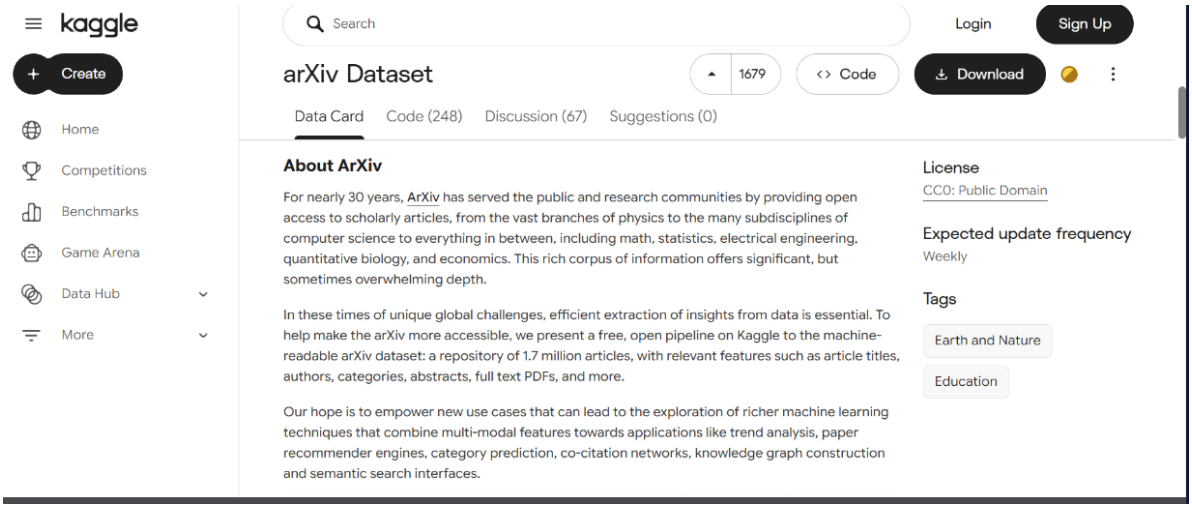
Team members:

#	Name	Roll No.	Main area
1	Harshit Pant	cs26mt006	Backend: all REST endpoints (resources, collections, roles and moderation), upload and chunking, AI retrieval pipeline
2	Snehadip Ghoshal	cs26mt010	Database (schema, migrations, seed data), frontend UI/UX for all pages, accessibility, Figma, Playwright E2E tests
3	Gangadevi Venkata Siva krishna	cs26mt009	OpenAPI spec, authentication and security, Docker/CI, AI guardrails, mocked AI tests and evaluation set
4	Ritik Kumar	cs26mt021	PPT and Testing

- **Description:** ScholarPulse is a multi-page academic knowledge hub where students upload, search, rate, comment on and organise course notes, paper summaries and reading lists. Teaching assistants publish official reading lists, and moderators review reported content. To reduce information overload during exam preparation, it includes an AI Study Assistant (Google Gemini API, called server-side) that answers questions using only the text of a chosen resource, or of a whole collection or course, and cites the exact passages it used. If the answer is not in the document, the assistant says so instead of guessing. If the AI service is slow, down or out of quota, the site falls back to keyword search that shows the matching passages, and every other feature keeps working.
- **Target users:** Undergraduate and postgraduate students looking for structured study material, teaching assistants who curate course reading lists, and moderators who keep the library clean. (Scope is deliberately limited to course-based study.)
- **How it differs from existing tools:** General AI tools with file upload (for example NotebookLM or ChatGPT with attachments) typically work on files that one user supplies. ScholarPulse is a shared, course-organised, moderated library with community ratings, and its AI answers are grounded in the library's own documents, can combine several documents of a course or reading list into one answer, and always show verifiable citations. The "not found" behaviour and the retrieval quality are measured in an evaluation and reported honestly.

2 Dataset

- **Primary source:** Cornell University arXiv Metadata Dataset on Kaggle, `https://www.kaggle.com/datasets/Cornell-University/arxiv`
- **Licence:** CC0: Public Domain (CC0 1.0), as shown in the License field of the dataset's Kaggle page (checked on 18 Oct 2026) and confirmed on a mirror dataset card. CC0 applies to the arXiv metadata (titles, abstracts, authors, categories), which is all this project uses; full-text PDFs are not used.



- **How the dataset is built:** The raw file is far too large to ship, so a one-time script (scripts/build_dataset.py) filters it to computer-science papers and samples about 600. The sampled subset is committed to the repository as data/seed/arxiv_subset.jsonl (a few MB), and the seed script loads that file, so a fresh clone works with docker compose up and no download. Each paper becomes a "paper summary" resource from its title and abstract. The team also writes about 30 original course-note documents (2 to 4 pages each) on algorithms, databases and operating systems; these are long enough for chunking and multi-passage questions and form the ground truth for the AI evaluation, which therefore runs on these notes and not on the abstracts. Users, ratings, comments, collections and a few sample reports are synthetic, generated with Faker and a fixed random seed.
- **Approximate size:** ~630 resources, ~20 users (including sample TA and moderator accounts), ~800 ratings, ~300 comments, ~15 collections.

Field mapping:

Source field	Where it goes and why
arXiv title	resources.title (indexed, VARCHAR 200)
arXiv abstract / team-written notes	resources.content_text, split into resource_chunks (with embeddings) for retrieval and Q&A
arXiv categories (e.g. cs.DB)	tags via resource_tags; a mapping table assigns each paper to a course code (CS301, CS402, CS205, SE101)
arXiv authors	resources.source_authors (plain text). Authors are not platform users.
arXiv id and update date	resources.source_ref and resources.published_on
Synthetic users	users (username, email, Argon2id password hash, bio, role: student / TA / moderator)
Synthetic reviews, lists, reports	ratings, comments, collections, collection_items, reports

3 Content Structure (Sitemap, 15 Pages)

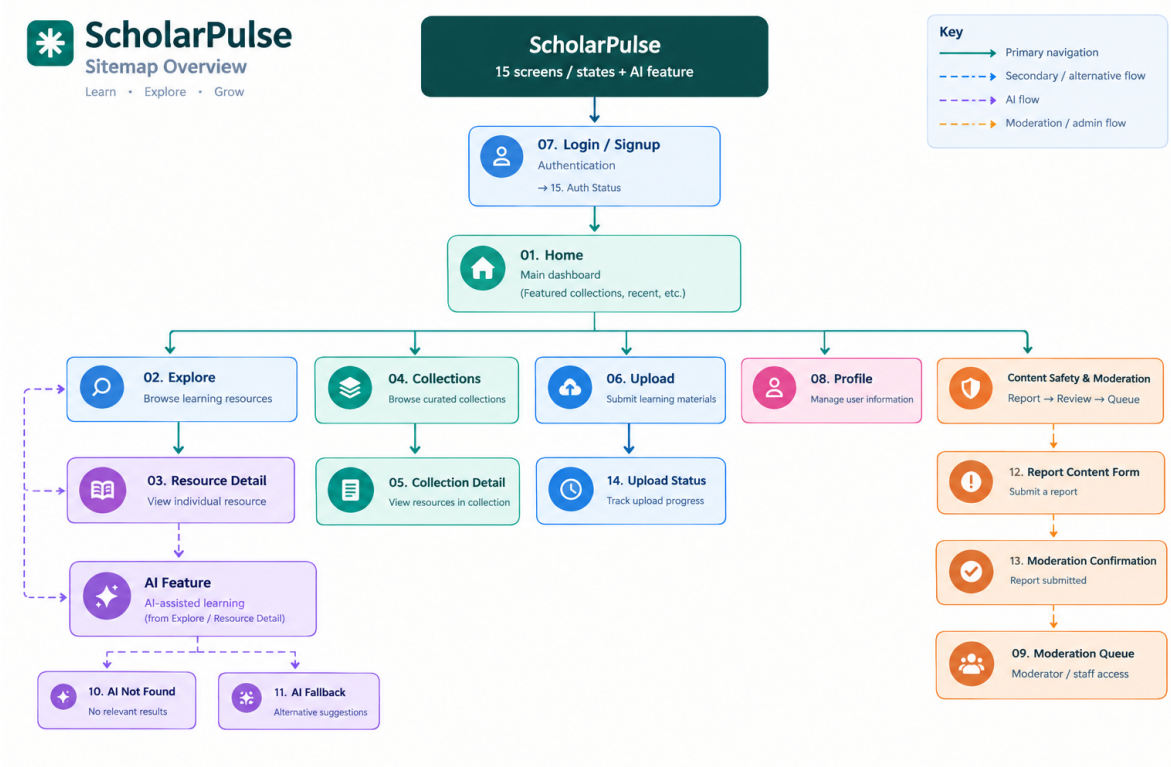


*Figure 1: Sitemap. * = login required (visitors are redirected to Auth); ** = moderator role only. Solid arrows = navigation; dashed arrows = conditional or action-based links.*

4 Functionality

4.1 Roles and permissions

Role	Can do
Visitor	Browse, search and read resources, comments and public collections.
Student	Everything a visitor can; upload notes; rate and comment; ask the AI; create collections; report content; edit and delete own content; delete own account.
TA	Everything a student can; publish official course reading lists (marked official and shown first); upload resources marked as official.
Moderator	Everything a student can; review the report queue; hide, restore or remove resources and comments; dismiss reports.

4.2 Page-by-page features

#	Page	What it shows / what the user can do
1	Home	Top-rated, newest and official resources, popular tags, course shortcuts, search box. Resources open directly in Resource detail. Visitors can use it without login.
2	Explore	Paginated catalogue with keyword search; filter by course code, tag and type; sort by rating, date or number of comments. When a course code is selected, an "Ask AI about this course" box answers across all resources of that course.
3	Resource detail	Title, text or PDF preview, tags, average rating, AI TL;DR summary, comments and star rating (login required), "Add to collection", "Report", and the Ask AI box. Answers show cited source passages, a "not found" message when applicable, and a notice when running in keyword fallback mode.
4	Collections hub	Public reading lists showing owner and size, with official TA lists first; logged-in users can create a list, and only TAs can mark one as official.
5	Collection detail	Ordered resources in a list; the owner can add, remove and reorder items and edit the title. An "Ask AI across this list" box answers questions using up to 20 resources of the list, with citations to each source.
6	Upload	Form for title, type, course code and tags, plus a file (PDF or .txt, maximum 5 MB) and a confirmation that the user has the right to share it. The server extracts the text, splits it into chunks, computes embeddings and generates a summary. A PDF with no extractable text (for example a scan) is rejected with a clear message.
7	Auth	Registration and login with a consent notice explaining what data is collected and why; logout. Passwords are stored as salted Argon2id hashes.
8	Profile	Edit username and bio, view my uploads, ratings and collections, change password, and delete the account together with all associated data.
9	Moderation queue (moderator only)	List of reports with reason and the reported item; hide, restore, remove or dismiss; each action is logged.

4.3 User stories

1. As a visitor, I want to browse and search study resources without creating an account so that I can judge whether the site is useful before I sign up.
2. As a student, I want to filter resources by course code, tag and type so that I find material for the exam I am preparing for.
3. As a student, I want to upload my class notes as a PDF or text file so that my classmates can access and use them.
4. As a student, I want to rate and comment on a resource so that reliable material rises

to the top and errors get flagged.

5. As a student, I want to ask a question about a resource and get an answer that cites the exact passage so that I can verify it instead of trusting the AI blindly.
6. As a student, I want to be told "not found" when the answer is not in the document so that I am not misled by a made-up answer.
7. As a student, I want to ask one question across all the resources of a collection or course so that I get an answer that combines several sources, with a citation to each.
8. As a student, I want a short summary of each resource so that I can decide quickly whether it is worth my time.
9. As a teaching assistant, I want to publish an official reading list for my course, marked as official, so that students know what to read first.
10. As a student, I want to save resources into my own collections so that I can organise my revision by subject.
11. As a student, I want to report an upload that is inappropriate or copied lecture material so that it can be reviewed and removed.
12. As a moderator, I want a queue of reported content where I can hide, remove or dismiss each report so that the library stays clean and respects copyright.
13. As a member, I want to edit or delete my own uploads and comments, and delete my account, so that I stay in control of what I share and of my personal information.

5 AI Feature Proposal: Grounded Study Assistant

- **Main feature:** Cited question answering over a resource (retrieval-augmented generation). The user asks a question; the server retrieves the most relevant chunks, asks the model to answer using only those chunks, and returns the answer with citations to the chunks used. If the retrieved text does not contain the answer, the response is "Not found in this document".
- **Secondary feature:** A short TL;DR summary generated once at upload and stored. It is built only after the main feature is tested. Automatic tag suggestion is out of scope.
- **Multi-document Q&A:** The same pipeline can answer across several documents. The user chooses the scope: a single resource (default), a collection (up to 20 resources) or a course code. Retrieval runs over the chunks of every resource in scope, takes the top 8 chunks with at most 3 per resource so that one long document cannot dominate, and the model must cite each chunk with its resource title. If sources disagree, the answer says so; if no chunk supports an answer, the response is still "Not found". Multi-document Q&A is a scope parameter of the single-document pipeline, so it is built on top of it and shares its guardrails and tests.
- **Models and API:** Google Gemini API (free tier), called from the server only. Embeddings: gemini-embedding-001 for chunks and questions. Answers and summaries: gemini-2.5-flash. Model names are kept in configuration so they can be changed without code changes if Google retires a model. The API key is stored in environment variables and GitHub Secrets and never appears in the code, repository or frontend.
- **Retrieval (decided):** Embedding-based. At upload, text is split into chunks of about 800 characters with 100 characters of overlap, and each chunk is embedded and stored in MySQL. At question time the question is embedded, cosine similarity is computed in the backend, and the top 5 chunks above a similarity threshold are used. Even a course scope holds only a few thousand chunks, so no separate vector database is needed. A MySQL

FULLTEXT index over the same chunks is the keyword baseline and the fallback.

- **Pipeline:** upload, validate, extract text, chunk, embed, store. Question: embed, retrieve top 5, send question and chunks to the model, validate that every cited chunk id was retrieved, return the answer with citations.
- **Data sent to the external service:** Only the user's question and the text of public resources (chunks). Emails, usernames, passwords and other personal identifiers are never sent. Free-tier terms: Google states that content sent on the Gemini API free tier may be used to improve its products (this does not apply in the UK, Switzerland and the EEA). For this reason, only public resource text and the question are sent, and nothing private is ever included.
- **Failure handling:** Each AI request has a 10-second timeout and one retry. On timeout, error or quota exhaustion, the user sees a clear message and the same page returns the best-matching passages from keyword search. Browsing, rating, commenting, collections, uploads and moderation are unaffected. If embedding fails at upload, the resource is still saved and its chunks are embedded later by a retry job. Summaries are generated in the background, so a failure never blocks an upload.
- **Guardrails:** Questions are limited to 300 characters and each user to 20 questions per hour. Document text is treated as untrusted data: it is wrapped in delimiters together with an instruction to ignore any commands found inside it. Model output is validated (cited chunk ids must have been retrieved) and displayed as escaped plain text. The model has no tools and no file or database access. Multi-document scope is capped at 20 resources and a fixed context size, and hidden or removed resources are never included. A document containing "ignore previous instructions" must not change the answer.
- **Testing:** The model and embedding clients are mocked in unit and integration tests, so CI never calls the external service or uses quota. Timeouts and errors are simulated to test the keyword fallback, which in multi-document mode shows matching passages grouped by resource.
- **Evaluation plan:** About 40 questions over the team-written notes, which are grouped into collections and courses: single-passage questions, questions needing two or more chunks, paraphrased questions (to compare embeddings with keyword search), at least 8 multi-document questions that need two or three different notes, at least 8 unanswerable questions (including some asked across a whole collection), and at least 3 poisoned documents with injected instructions. Retrieval and refusal are measured separately: (1) retrieval recall at 5, i.e. whether the correct chunk was retrieved, for embeddings versus keyword baseline; (2) correct-refusal rate on unanswerable questions; (3) answer correctness and citation accuracy on answerable ones; (4) for multi-document questions, source coverage (whether every needed document was retrieved and cited) and whether any citation points to the wrong source. Failures and their causes will be reported honestly.

6 Security and Privacy Plan

- Argon2id salted password hashes; passwords never logged or returned by any API.
- Authentication uses server-side session cookies (HttpOnly, SameSite=Lax, Secure in production, 24-hour expiry) with CSRF protection on state-changing requests. Protected endpoints return 401 or 403 and tests prove it.
- Role checks on every protected endpoint (student, TA, moderator); users can edit or delete only their own content; only moderators can hide or remove others' content.

- ORM or parameterised queries only; server-side validation on every input; uploads restricted to PDF or .txt and 5 MB; extracted text is escaped before display.
- AI scope safety: moderator-hidden or removed resources are excluded from retrieval and from model context; scope size is capped.
- Copyright and abuse: uploaders confirm they have the right to share the file, any user can report content, and moderators can hide or remove it with a logged action.
- DPDP Act 2023 spirit: collect only needed data, consent notice at signup, account and data deletion from the profile page.
- The final report documents a review against the OWASP Top 10 and the OWASP Top 10 for LLM Applications.

7 Architecture and Technical Plan

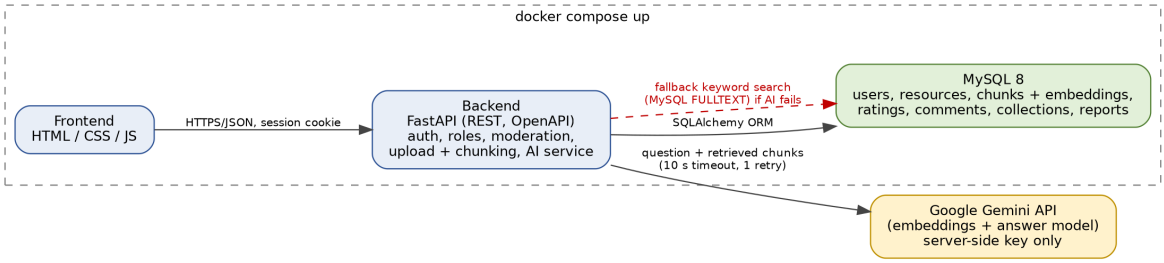


Figure 2: Architecture. The whole stack starts with one docker compose up; the AI service is the only external dependency.

- **Stack:** FastAPI, SQLAlchemy and Alembic, MySQL 8, plain HTML/CSS/JS frontend, Docker Compose, pytest and pytest-cov, Playwright, GitHub Actions with a MySQL service container, Ruff.
- **Database (planned 12 tables):** users (with role), categories, resources, resource_chunks (with embedding), tags, resource_tags, ratings, comments, collections, collection_items, reports, ai_queries (rate limiting and audit, with scope type and scope id). The full ER diagram follows in MS-2.
- **Requirements coverage:** REST API (OpenAPI, FastAPI), MySQL with 5+ related tables, authentication, 9 pages (6 required), one AI feature, and a single docker compose up for the whole stack.

8 Timeline, Responsibilities and Risks

8.1 Timeline

Milestone	Date	Main work
MS-1	15 Oct	Design document (this revision) and repository setup
MS-2	22 Oct	Figma prototype (login, Ask AI, report and moderation screens), accessibility check, ER diagram, index notes
MS-3	29 Oct	OpenAPI YAML, migrations, seed script with committed subset, docker compose up for backend and database
MS-4	5 Nov	pytest and CI, 60% coverage, frontend for browse, search and view, reviewed PRs
MS-5	12 Nov	Auth, roles, uploads, moderation, single-document AI Q&A, then multi-document scope, with mocks and the evaluation set, 2 Playwright tests, 70% coverage
MS-6	19 Nov	Final report, presentation, demo and viva

8.2 Who builds what

Member	Responsibilities
Harshit Pant	All backend: endpoints for resources, collections, roles and moderation (reports, hide/restore/remove, action log), upload and chunking, embedding and retrieval pipeline including multi-document scopes (collection and course).
Snehadip Ghoshal	Database (schema, Alembic migrations, indexes, ER diagram, seed script and dataset subset). Figma prototype, all frontend pages (including Auth, Moderation queue, the AI scope selector and the fallback and not-found states), accessibility, connecting the frontend to the API, Playwright end-to-end tests.
Gangadevi Venkata Siva krishna	OpenAPI specification, authentication, sessions and CSRF, security checklist, Docker and CI, AI guardrails, mocked AI tests and the evaluation set (single and multi-document).
Ritik Kumar	PPT presentation and testing (pytest suite and coverage, manual QA).

Every member reviews another member's pull requests, and every member must be able to explain any part of the code in the viva.

8.3 Risks

Risk	Why it matters	Mitigation
Gemini quota or outage	Q&A stops working	10 s timeout, one retry, keyword fallback, mocked tests
Paraphrased questions missed	"Not found" could reflect retrieval failure, not real absence	Embedding retrieval; evaluate retrieval and refusal separately
Scanned or empty PDFs	No text to index	Reject files with no extractable text, with a clear message
Copyrighted uploads	Legal and trust problem	Upload confirmation, report button, moderator removal
Evaluation set takes long to write	Weak AI marks	Start the 30 notes and questions by MS-3
Multi-document answers mix up sources	Wrong or missing citations	Cap of 3 chunks per resource, citation per source, dedicated multi-document evaluation questions
Scope for 3 people	Missed milestone costs 10%	Multi-document is a scope parameter on the single-document pipeline (build single first); freeze features after MS-5

9 Repository Setup

- GitHub repository created: cs26mt010-tech/scholarpulse
- All 4 members added as collaborators
- README drafted: name, description, tech stack, docker compose up steps
- Branch protection enabled on main: changes require a pull request reviewed by at least one other team member
- AI_USAGE.md created; .env.example added; no API keys committed